



Python+DS
fwdays

FEDERATED LEARNING: OUT OF REACH NO MATTER HOW CLOSE

Oleksandr Lapshyn
Apheris AI GmbH

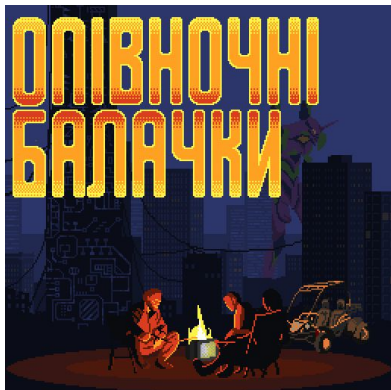
About me

Berlin based

Software Engineer at Apheris AI GmbH

[@sudodoki](#) in most social media

podcast Опівночні Балачки



Distributed training. Ranging from...

- single instance on single dataset
- data parallel
- same company, cross departments
- different companies
- edge device training

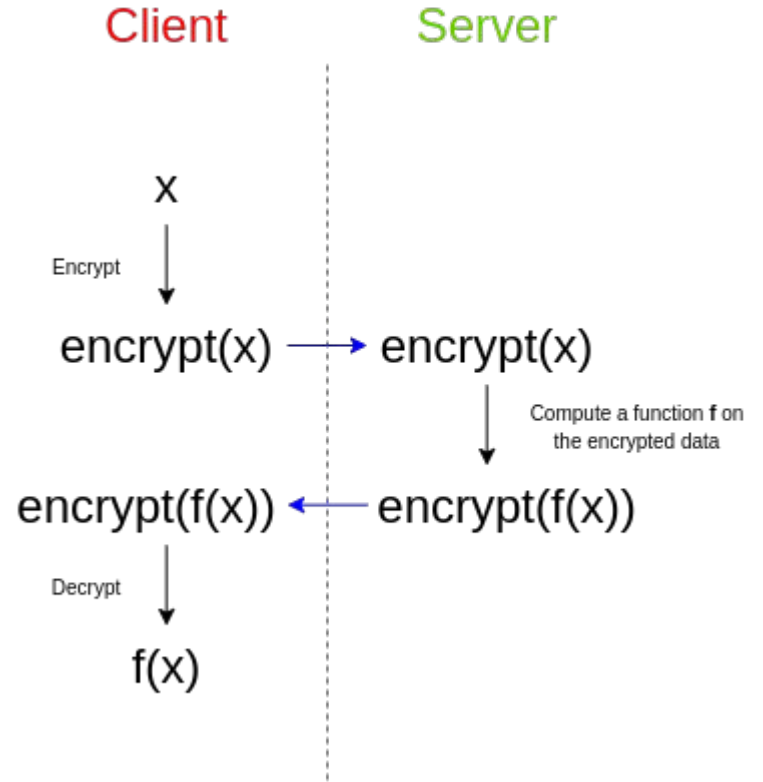
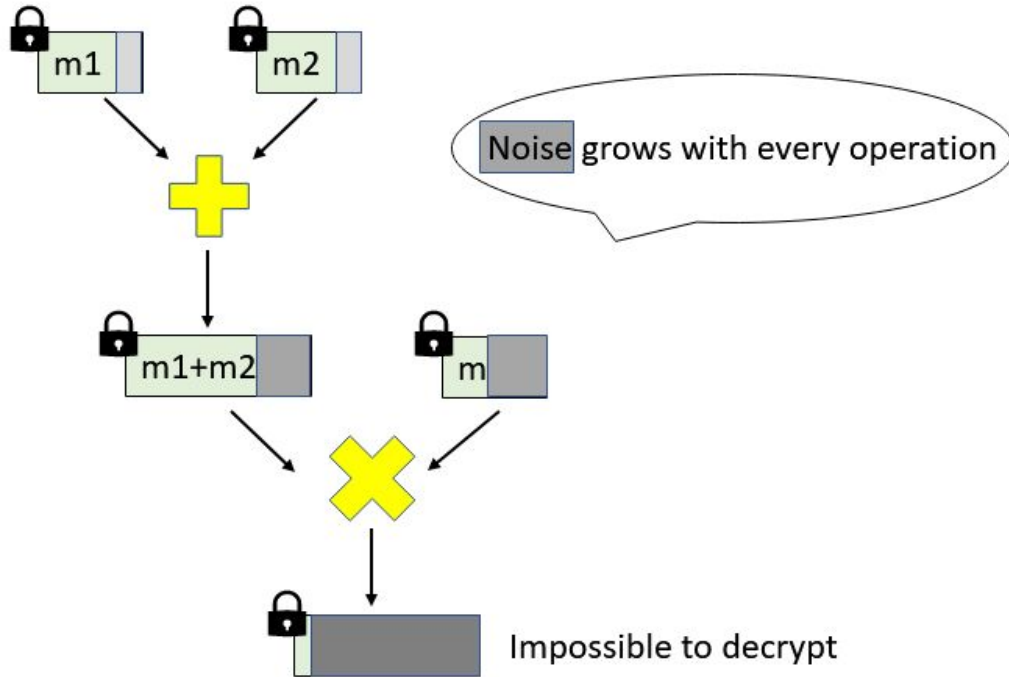
Reasons to use federated learning setup

- legal reasons: your data cannot be shared with 3rd parties, GDPR, HIPAA ...
 - proprietary data: your data is your IP
 - saving money: not moving data from storage to where computations are made
 - possible data value space is huge / class disbalance is huge and you want as much data as possible: structural biology, exotic diseases, etc
- ... something else I'm forgetting

FL most likely resulting in:

- Federated Statistics / Analytics
- Machine Learning

My idea of FL as of 2012



My idea of FL as of 2024

Homomorphic Encryption is cool but not practical:

- performance setbacks
- unbounded growth of data being send back and forth

Still ton of cool research in areas of Secure Multiparty Computation, Secure Aggregation Algorithms

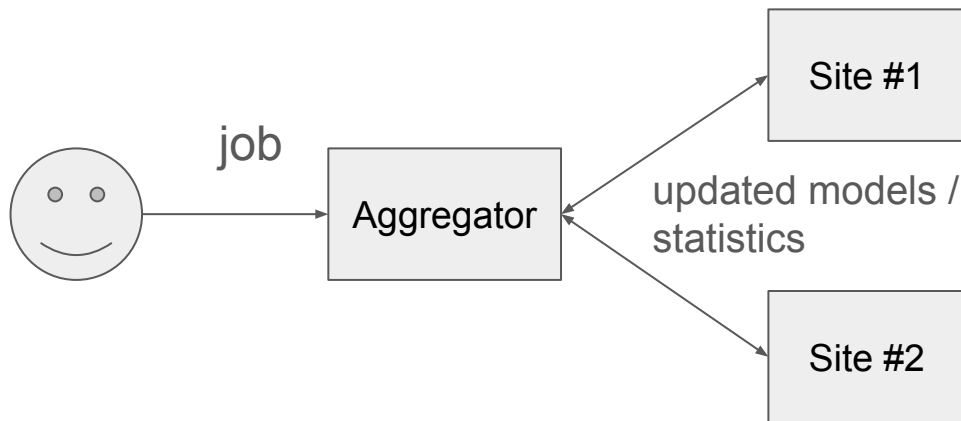
Engineers



Lawyers

What does it take to do cross-org FL?

Using centralized aggregator topology



Engineering tools to keep data safe

- Differential Privacy aka 'just add some noise'
- Privacy Preserving techniques, like filtering etc
- RBAC & Audit trail



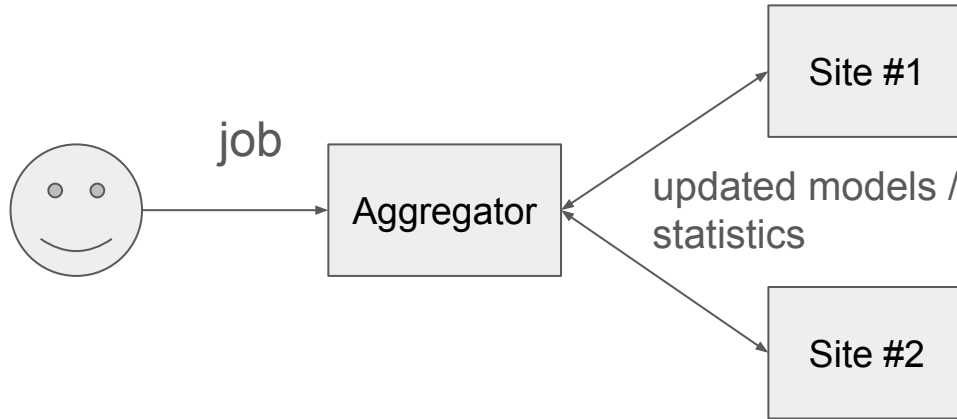
FL and EU AI Act

- GDPR is still applicable: end user can revoke permission to use their data
- checking data for possible bias
- we should avoid data leak ➡ training process should guard against possible model attacks (gradient inversion, membership inference)
- Art. 69 regarding CoC for environmental sustainability

Lot's of solutions in the space

- [PySyft + PyGrid](#)
- [FATE](#)
- [flower](#)
- [FedML](#)
- [NVFlare](#)
- [DataSHIELD](#)
- others, both open-source & proprietary

What does it take to support cross-org FL?



- Authentication using root certificate authority
- Org/role based authorization
- Job scheduling
- Client registering, heartbeats, resource allocation
- Custom serialization protocol to avoid pickle's CVE
- DSL for jobs

Still so much is left out of the picture

- Getting data between orgs to be of same format OR being able to run diff preprocessing
- Data skew/imbalance, site-specific training regimen
- Choosing optimal aggregation strategy and frequency

Possible research areas / grow zones [not all FL-specific]

- HE tailored hardware
- Optimal aggregation strategies
- Mitigating ML models attack vectors
- Standardized data formats and adopting those
- Making FLOps a thing outside the bubble
 - how to manage data / model access
 - deploying the stuff
 - making sure it runs and works [we are still talking distributed cross org computations]

FL is here, and it will stay

It might be niche

It might be yet not 100% mature

But there is fun in it, try it out

Further reading

1. H. Woisetschläger, *Federated Learning Priorities Under the EU AI Act* @ [arxiv](#)
2. C. Gentry, *Fully homomorphic encryption using ideal lattices* [article](#), [thesis version](#)
3. K. Bonawitz, *Practical Secure Aggregation for Privacy-Preserving Machine Learning*, [dl.acm.org](#)
4. M. Roehm, *Top 7 Open-Source Frameworks for Federated Learning*, [blogpost](#)
5. [Flower Tutorial](#)
6. [NVFlare examples](#)

You made it to the final slide

For questions use Discord, tg
or ping me at @sudodoki



http://t.me/volunteer_diary

Addendum

Install

```
$> pip install nvflare==2.4.0
```

```
Successfully installed Flask-2.2.5 Flask-JWT-Extended-4.4.3  
Flask-SQLAlchemy-2.5.1 Jinja2-3.1.3 MarkupSafe-2.1.5  
PyJWT-2.8.0 PyYAML-6.0.1 SQLAlchemy-1.4.31 Werkzeug-2.2.2  
certifi-2024.2.2 cffi-1.16.0 charset-normalizer-3.3.2  
click-8.1.7 cryptography-42.0.5 docker-7.0.0 greenlet-3.0.3  
grpcio-1.51.1 gunicorn-22.0.0 idna-3.7 itsdangerous-2.2.0  
msgpack-1.0.8 numpy-1.26.4 nvflare-2.4.0 packaging-24.0  
protobuf-3.20.3 psutil-5.9.8 pycparser-2.22 pyhocon-0.3.60  
pyparsing-3.1.2 requests-2.31.0 six-1.16.0 urllib3-2.2.1  
websockets-12.0
```

Bootstrap project.yml

```
$> nvflare provision
```

No project.yml found in current folder.

There are two types of templates for project.yml.

1) project.yml for HA mode

2) project.yml for non-HA mode

3) Don't generate project.yml. Exit this program.

Handcrafted project.yml

```
api_version: 3
name: example_project
description: NVIDIA FLARE sample project yaml file
```

participants:

```
- name: overseer
  type: overseer
  org: fwdays_demo
  protocol: https
  api_root: /api/v1
  port: 8443

- name: server1.my-domain.example
  type: server
  org: fwdays_demo
  fed_learn_port: 8002
  admin_port: 8003

- name: site-1
  type: client
  org: fwdays_demo

- name: admin@giving-talks.com
  type: admin
  org: fwdays_demo
  role: project_admin
```

builders:

```
- path: nvflare.lighter.impl.workspace.WorkspaceBuilder
  args:
    template_file: master_template.yml

- path: nvflare.lighter.impl.template.TemplateBuilder
  # next 2 from HA mode

- path: nvflare.lighter.impl.docker.DockerBuilder
  args:
    base_image: python:3.8
    requirements_file: docker_compose_requirements.txt

- path: nvflare.lighter.impl.helm_chart.HelmChartBuilder
  args:
    docker_image: localhost:32000/nvfl-min:0.0.1

- path: nvflare.lighter.impl.static_file.StaticFileBuilder
  args:
    config_folder: config
    overseer_agent:
      path: nvflare.ha.overseer_agent.HttpOverseerAgent
      overseer_exists: true

- path: nvflare.lighter.impl.cert.CertBuilder

- path: nvflare.lighter.impl.signature.SignatureBuilder
```

running nvflare provision again

```
$> tree ./workspace/example_project/prod_00
```

```
├── admin@giving-talks.com
│   ├── local
│   ├── startup
│   │   ├── client.crt
│   │   ├── client.key
│   │   ├── client.pfx
│   │   ├── fed_admin.json
│   │   ├── fl_admin.sh
│   │   ├── readme.txt
│   │   └── rootCA.pem
│   └── transfer
├── compose.yaml
├── nvflare_compose
│   ├── Dockerfile
│   └── requirements.txt
├── nvflare_hc
│   ├── Chart.yaml
│   ├── templates
│   │   ├── deployment_overseer.yaml
│   │   ├── deployment_server0.yaml
│   │   ├── service_overseer.yaml
│   │   └── service_server0.yaml
│   └── values.yaml
├── overseer
│   ├── local
│   ├── startup
│   │   ├── gunicorn.conf.py
│   │   ├── overseer.crt
│   │   ├── overseer.key
│   │   ├── overseer.pfx
│   │   ├── privilege.yml
│   │   ├── rootCA.pem
│   │   ├── signature.json
│   │   └── start.sh
│   └── transfer
```

```
├── server1.my-domain.example
│   ├── local
│   │   ├── authorization.json.default
│   │   ├── log.config.default
│   │   ├── privacy.json.sample
│   │   └── resources.json.default
│   ├── readme.txt
│   ├── startup
│   │   ├── fed_server.json
│   │   ├── rootCA.pem
│   │   ├── server.crt
│   │   ├── server.key
│   │   ├── server.pfx
│   │   ├── signature.json
│   │   ├── start.sh
│   │   ├── stop_fl.sh
│   │   └── sub_start.sh
│   └── transfer
├── site-1
│   ├── local
│   │   ├── authorization.json.default
│   │   ├── log.config.default
│   │   ├── privacy.json.sample
│   │   └── resources.json.default
│   ├── readme.txt
│   ├── startup
│   │   ├── client.crt
│   │   ├── client.key
│   │   ├── client.pfx
│   │   ├── fed_client.json
│   │   ├── rootCA.pem
│   │   ├── signature.json
│   │   ├── start.sh
│   │   ├── stop_fl.sh
│   │   └── sub_start.sh
│   └── transfer
```

compose.yml [minified 'command']

```
services:
  overseer:
    build: nvflare_compose
    command:
      - ${WORKSPACE}/startup/start.sh
    container_name: overseer
    image: ${IMAGE_NAME}
    ports:
      - 8443:8443
    volumes:
      - ./overseer:${WORKSPACE}
  server1.my-domain.example:
    command:
      - ${PYTHON_EXECUTABLE} -u -m nvflare.private.fed.app.server.server_train -m ${WORKSPACE} -s fed_server.json --set secure_train=true config_folder=config org=fwdays_demo
    container_name: server1.my-domain.example
    image: ${IMAGE_NAME}
    ports:
      - 8002:8002
      - 8003:8003
    volumes:
      - ./server1.my-domain.example:${WORKSPACE}
      - nvflare_svc_persist:/tmp/nvflare/
  site-1:
    command:
      - ${PYTHON_EXECUTABLE} -u -m nvflare.private.fed.app.client.client_train -m ${WORKSPACE} -s fed_client.json --set secure_train=true uid=site-1 org=fwdays_demo config_folder=config
    container_name: site-1
    image: ${IMAGE_NAME}
    volumes:
      - ./site-1:${WORKSPACE}
volumes:
  nvflare_svc_persist: null
```

helm template workspace/example_project/prod_00/nvflare_hc/

```
# Source: nvflare/templates/service_overseer.yaml
apiVersion: v1
kind: Service
metadata:
  name: overseer
spec:
  ports:
    - port: 8443
      protocol: TCP
      targetPort: 8443
  selector:
    system: overseer
---
# Source: nvflare/templates/service_server0.yaml
apiVersion: v1
kind: Service
metadata:
  labels:
    system: server1.my-domain.example
  name: server1.my-domain.example
spec:
  ports:
    - name: fl-port
      port: 8002
      protocol: TCP
      targetPort: 8002
    - name: admin-port
      port: 8003
      protocol: TCP
      targetPort: 8003
  selector:
    system: server1.my-domain.example
```

```
# Source: nvflare/templates/deployment_overseer.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    system: overseer
  name: overseer
spec:
  replicas: 1
  selector:
    matchLabels:
      system: overseer
  template:
    metadata:
      labels:
        system: overseer
    spec:
      containers:
        - command:
            - /workspace/overseer/startup/start.sh
          image: localhost:32000/nvfl-min:0.0.1
          imagePullPolicy: IfNotPresent
          name: overseer
          ports:
            - containerPort: 8443
              name: overseer-port
              protocol: TCP
          volumeMounts:
            - mountPath: /workspace
              name: workspace
      volumes:
        - hostPath:
            path: '/home/nvflare'
            type: Directory
          name: workspace
```

```
# Source: nvflare/templates/deployment_server0.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    system: server1.my-domain.example
  name: server1.my-domain.example
spec:
  replicas: 1
  selector:
    matchLabels:
      system: server1.my-domain.example
  template:
    metadata:
      labels:
        system: server1.my-domain.example
    spec:
      containers:
        - args:
            - -u
            - -m
            - nvflare.private.fed.app.server.server_train
            - -m
            - /workspace/server1.my-domain.example
            - -s
            - fed_server.json
            - --set
            - secure_train=true
            - config_folder=config
            - org=fwdays_demo
          command:
            - /usr/local/bin/python3
          image: localhost:32000/nvfl-min:0.0.1
          imagePullPolicy: IfNotPresent
          name: server1.my-domain.example
          ports:
            - containerPort: 8002
              protocol: TCP
            - containerPort: 8003
              protocol: TCP
          volumeMounts:
            - mountPath: /workspace
              name: workspace
            - mountPath: /tmp/nvflare
              name: persist
      volumes:
        - hostPath:
            path: '/home/nvflare'
            type: Directory
          name: workspace
        - hostPath:
            path: '/home/nvflare'
            type: Directory
          name: persist
```

Sample fed_server.json

```
{
  "format_version": 2,
  "servers": [
    {
      "name": "example_project",
      "service": {
        "target": "server1.my-domain.example:8002",
        "scheme": "grpc"
      },
      "admin_host": "server1.my-domain.example",
      "admin_port": 8003,
      "ssl_private_key": "server.key",
      "ssl_cert": "server.crt",
      "ssl_root_cert": "rootCA.pem"
    }
  ],
  "overseer_agent": {
    "path": "nvflare.ha.overseer_agent.HttpOverseerAgent",
    "args": {
      "role": "server",
      "overseer_end_point": "https://overseer:8443/api/v1",
      "project": "example_project",
      "name": "server1.my-domain.example",
      "fl_port": "8002",
      "admin_port": "8003"
    }
  }
}
```

Sample Job's app/config/config_fed_client.json

```
{
  "format_version": 2,
  "executors": [
    {
      "tasks": [
        "train",
        "submit_model"
      ],
      "executor": {
        "path": "nvflare.app_common.np.np_trainer.NPTrainer",
        "args": {}
      }
    },
    {
      "tasks": [
        "validate"
      ],
      "executor": {
        "path": "nvflare.app_common.np.np_validator.NPValidator"
      }
    }
  ],
  "task_result_filters": [],
  "task_data_filters": [],
  "components": []
}
```

Sample Job's app/config/config_fed_server.json

```
{
  "format_version": 2,
  "server": {
    "heart_beat_timeout": 600
  },
  "task_data_filters": [],
  "task_result_filters": [],
  "components": [
    {
      "id": "persistor",
      "path": "nvflare.app_common.np.np_model_persistor.NPModelPersistor",
      "args": {}
    },
    {
      "id": "shareable_generator",
      "path": "nvflare.app_common.shareablegenerators.full_model_shareable_generator.FullModelShareableGenerator",
      "args": {}
    },
    {
      "id": "aggregator",
      "path": "nvflare.app_common.aggregators.intime_accumulate_model_aggregator.InTimeAccumulateWeightedAggregator",
      "args": {
        "expected_data_kind": "WEIGHTS",
        "aggregation_weights": {
          "site-1": 1.0,
          "site-2": 1.0
        }
      }
    },
    {
      "id": "model_locator",
      "path": "nvflare.app_common.np.np_model_locator.NPModelLocator",
      "args": {}
    }
  ],
  "workflows": [
    {
      "id": "scatter_and_gather",
      "path": "nvflare.app_common.workflows.scatter_and_gather.ScatterAndGather",
      "args": {
        "min_clients": 2,
        "num_rounds": 3,
        "start_round": 0,
        "wait_time_after_min_received": 10,
        "aggregator_id": "aggregator",
        "persistor_id": "persistor",
        "shareable_generator_id": "shareable_generator",
        "train_task_name": "train",
        "train_timeout": 6000
      }
    },
    {
      "id": "cross_site_model_eval",
      "path": "nvflare.app_common.workflows.cross_site_model_eval.CrossSiteModelEval",
      "args": {
        "model_locator_id": "model_locator",
        "submit_model_timeout": 600,
        "validation_timeout": 6000,
        "cleanup_models": false
      }
    }
  ]
}
```

```
{
  "id": "formatter",
  "path": "nvflare.app_common.np.np_formatter.NPFormatter",
  "args": {}
},
{
  "id": "json_generator",
  "path": "nvflare.app_common.widgets.validation_json_generator.ValidationJsonGenerator",
  "args": {}
},
{
  "id": "scatter_and_gather",
  "path": "nvflare.app_common.workflows.scatter_and_gather.ScatterAndGather",
  "args": {
    "min_clients": 2,
    "num_rounds": 3,
    "start_round": 0,
    "wait_time_after_min_received": 10,
    "aggregator_id": "aggregator",
    "persistor_id": "persistor",
    "shareable_generator_id": "shareable_generator",
    "train_task_name": "train",
    "train_timeout": 6000
  }
},
{
  "id": "cross_site_model_eval",
  "path": "nvflare.app_common.workflows.cross_site_model_eval.CrossSiteModelEval",
  "args": {
    "model_locator_id": "model_locator",
    "submit_model_timeout": 600,
    "validation_timeout": 6000,
    "cleanup_models": false
  }
}
]
```